

Fonctions Alarme & Timestamp sous Nios II

sys/alt_alarm.h

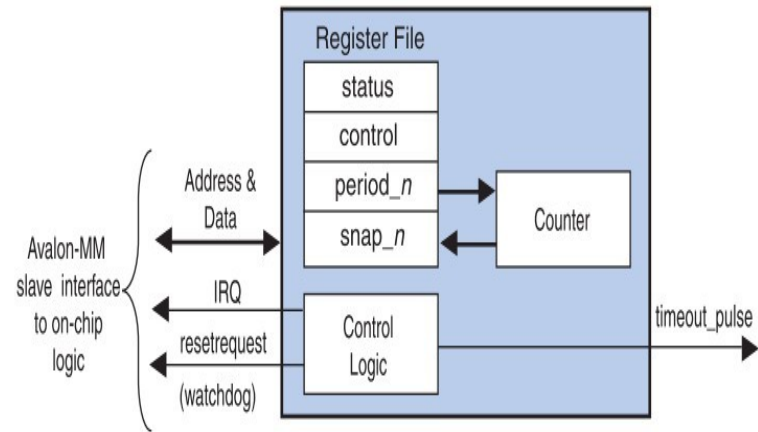
sys/alt_timestamp.h

Avalon IP Core Slave - Interval timer core

Modèle de programmation

- Taille registre : 16 bits
- 6 registres si compteur 32-bits
- 10 registres si compteur 64-bits

Offset	Name	R/W	Description of Bits					
			15	...	4	3	2	1 0
0	status	RW	(1)					RUN TO
1	control	RW	(1)		STOP	START	CONT	ITO
2	periodl	RW	Timeout Period – 1 (bits [15:0])					
3	periodh	RW	Timeout Period – 1 (bits [31:16])					
4	snapl	RW	Counter Snapshot (bits [15:0])					
5	snaph	RW	Counter Snapshot (bits [31:16])					



Remarque

Pour obtenir l'adresse d'un registre (si usage en bare metal), l'offset doit être multiplié par 4 et ajouté à l'adresse BASE ADDRESS (taille bus NIOS II adress Avalon 32-bits)

Avalon IP Core Slave – Exemple de configuration d'une l'instance via QSys

Paramètres clés

Composant	altera_avalon_timer
Type	Interval_timer
Timeout	125 ms
Compteur	32 bits
Preset	Custom

Registres

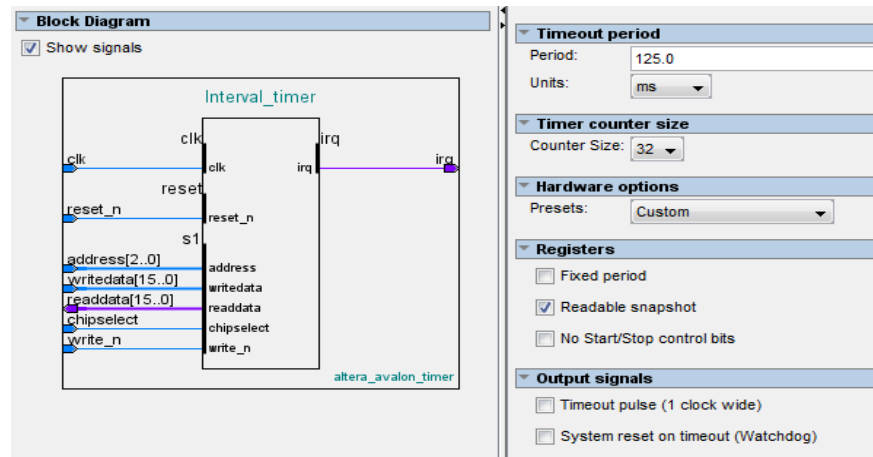
- Période fixe : désactivée
- Snapshot lisible : activé
- Bits Start/Stop : absents

Interface Avalon

- clk
- reset_n
- address[2..0]
- writedata[15..0]
- readdata[15..0]
- chipselect
- write_n

Sorties

- Impulsion Timeout : désactivée
- Reset système (Watchdog) : désactivé



Alarme (CONT = 0)

`sys/alt_alarm.h`

- ▶ Tâches périodiques via callbacks
- ▶ Timer s'arrête à zéro, se recharge
- ▶ Période = multiple du tick d'horloge
- ▶ Plusieurs alarmes simultanées
- ▶ Piloté par **system clock driver**

`altera_avalon_timer_sc.c`

Timestamp (CONT = 1)

`sys/alt_timestamp.h`

- ▶ Mesure d'intervalles haute résolution
- ▶ Compteur s'active en continu
- ▶ **Benchmarking de fonctions**
- ▶ Résolution = période horloge
- ▶ Piloté par **timestamp driver**

`altera_avalon_timer_tc.c`

Fonctions Alarme — `sys/alt_alarm.h`

`alt_alarm_start()`

```
int alt_alarm_start(alt_alarm* alarm, alt_u32 nticks, alt_u32(*callback)(void*), void* context)
```

Enregistre une callback appelée tous les nticks. Retourne 0 si succès. La callback retourne le délai (en ticks) jusqu'au prochain appel.

`alt_alarm_stop()`

```
void alt_alarm_stop(alt_alarm* alarm)
```

Arrête et désactive la callback associée à l'alarme. La structure alt_alarm peut ensuite être réutilisée.

`alt_ticks_per_second()`

```
alt_u32 alt_ticks_per_second(void)
```

Retourne le nombre de ticks par seconde du system clock. Utile pour exprimer des durées en secondes plutôt qu'en ticks bruts.

`alt_nticks()`

```
alt_u32 alt_nticks(void)
```

Retourne le nombre de ticks écoulés depuis le reset. Retourne 0 si aucun system clock n'est disponible dans system.h.

Fonctions Timestamp — `sys/alt_timestamp.h`

`alt_timestamp_start()`

```
int alt_timestamp_start(void)
```

Démarre le compteur timestamp (CONT=1, continu). Retourne 0 si succès, négatif si aucun timer timestamp disponible dans la config BSP.

`alt_timestamp()`

```
alt_u32 alt_timestamp(void)
```

Retourne la valeur courante du compteur en ticks. La différence entre deux appels donne la durée écoulée (compteur descendant).

`alt_timestamp_freq()`

```
alt_u32 alt_timestamp_freq(void)
```

Retourne la fréquence d'incrément du compteur en Hz. Conversion : durée (s) = delta_ticks / alt_timestamp_freq().

Fonctions UNIX (basées sur system clock) :

Gettimeofday() - settimeofday() - times()
T0 → depuis Epoch 1970

Structure de données :

```
struct timeval {  
    time_t tv_sec;  
    suseconds_t tv_usec;}
```

Exemple — Système d'alarme périodique

```
#include <sys/alt_alarm.h>
#include <stdio.h>

alt_alarm ma_structure_alarm;

// Fonction de rappel
alt_u32 ma_fonction_callback(void* context) {
    printf("Alarme déclenchée !\n");
    return alt_ticks_per_second(); // Se relance toutes les 1s
}

int main() {
    // Lance l'alarme pour la première fois dans 2 secondes
    if (alt_alarm_start(&ma_structure_alarm,
                       2 * alt_ticks_per_second(),
                       ma_fonction_callback,
                       NULL) < 0) {
        printf("Erreur : Pas de timer système configuré.\n");
    }
    while(1); // Code continue ici - Alarme en arrière-plan
    Return 0;
}
```

1

Callback -> (alarm_callback(void* ctx)

Appelée à chaque tick. Retourne le délai en ticks jusqu'au prochain appel.

2

alt_ticks_per_second()

Exprime le délai en secondes plutôt qu'en ticks bruts.

3

alt_alarm_start()

Enregistre l'alarme. Elle se déclenche après nticks puis à chaque retour de callback.

Exemple — Mesure de temps d'exécution

```
#include "sys/alt_timestamp.h"
#include "alt_types.h"
#include <stdio.h>
#include <unistd.h> // Pour la fonction usleep()

int main(void) {
    alt_u32 t1, t2;
    float dt;

    // Initialisation du compteur (Timestamp)
    if (alt_timestamp_start() < 0) {
        printf("Erreur: Pas de timer de type 'timestamp' consulter BSP \n");
        return -1;
    }

    // Mesure nombre de ticks
    t1 = alt_timestamp();
    usleep(); // Simulation fonction
    t2 = alt_timestamp();

    // Calcul du temps en secondes : (Ticks / Frequence)
    // On convertit en float pour obtenir la précision décimale
    dt = (float)(t2 - t1) / (float)alt_timestamp_freq();

    // Affichage des résultats
    printf("Frequence du timer:%uHz\n", (unsigned int)alt_timestamp_freq());
    printf("Ticks f1 = %u \n", (unsigned int)(t2 - t1));
    printf("dt = %.6f s\n", dt);

    return 0;
}
```

1

alt_timestamp_start()

Démarre le compteur. Toujours vérifier le retour avant de mesurer.

2

Relever t1, t2

Capturer la valeur avant et après chaque fonction à mesurer.

3

Calcul de durée

$\text{durée (s)} = (t2 - t1) / \text{alt_timestamp_freq()}$.
Compteur descendant.

4

system.h requis

ALT_TIMESTAMP_CLK doit pointer vers un timer (pas 'none') dans le BSP.

Exemple — Mesure de temps d'exécution

```
#include "sys/alt_timestamp.h"
#include "alt_types.h"
#include <stdio.h>
#include <unistd.h> // Pour la fonction usleep()

int main(void) {
    alt_u32 t1, t2;
    float dt;

    // Initialisation du compteur (Timestamp)
    if (alt_timestamp_start() < 0) {
        printf("Erreur: Pas de timer de type 'timestamp' consulter BSP \n");
        return -1;
    }

    // Mesure nombre de ticks
    t1 = alt_timestamp();
    usleep(); // Simulation fonction
    t2 = alt_timestamp();

    // Calcul du temps en secondes : (Ticks / Frequence)
    // On convertit en float pour obtenir la précision décimale
    dt = (float)(t2 - t1) / (float)alt_timestamp_freq();

    // Affichage des résultats
    printf("Frequence du timer:%uHz\n", (unsigned int)alt_timestamp_freq());
    printf("Ticks f1 = %u \n", (unsigned int)(t2 - t1));
    printf("dt = %.6f s\n", dt);

    return 0;
}
```

1

alt_timestamp_start()

Démarre le compteur. Toujours vérifier le retour avant de mesurer.

2

Relever t1, t2

Capturer la valeur avant et après chaque fonction à mesurer.

3

Calcul de durée

$\text{durée (s)} = (t2 - t1) / \text{alt_timestamp_freq()}$.
Compteur descendant.

4

system.h requis

ALT_TIMESTAMP_CLK doit pointer vers un timer (pas 'none') dans le BSP.

Exemple — Génération d'un signal PWM

```
#include <stdio.h>
#include "system.h"
#include "sys/alt_alarm.h"
#include "altera_up_avalon_parallel_port.h"
// Paramètres du signal
#define PERIOD_TP 1000
#define ACTIVE_TA 300

alt_alarm pwm_alarm;
alt_up_parallel_port_dev *led_port; // Descripteur du périphérique

//-----

// Callback unique avec gestion par Driver BSP
//-----

alt_u32 pwm_callback(void* context) {
    static int state = 0;

    if (state == 0) {
        // Transition BAS -> HAUT
        alt_up_parallel_port_write_data(led_port, 0x01);
        state = 1;
        return (alt_u32)ACTIVE_TA;
    } else {
        // Transition HAUT -> BAS
        alt_up_parallel_port_write_data(led_port, 0x00);
        state = 0;
        return (alt_u32)(PERIOD_TP - ACTIVE_TA);
    }
}
```

```
int main() {
    // 1. Ouverture du périphérique sachant BSP
    led_port=alt_up_parallel_port_open_dev(
        "/dev/parallel_port_0");

    if (led_port == NULL) {
        printf("Erreur : Impossible d'ouvrir
            le port parallele\n");
        return -1;
    }

    // 2. Lancement de la génération du PWM
    alt_alarm_start(&pwm_alarm, 10, &pwm_callback, NULL);

    while (1) ;

    return 0;
}
```

Exemple — Temps système

```
#include <stdio.h>
#include <sys/time.h>      // Pour struct timeval et settimeofday
#include <alt_types.h>     // Types spécifiques Altera
#include <unistd.h>        // Pour sleep()

int main(void)
{
    struct timeval now;
    struct timeval tv_verif;
    int rc;
    printf("--- Configuration de l'heure système (Nios II) ---\n");
    /* Initialisation timeval : * 4jours= 4*86400s, *23heures=23*3600s*58minutes=58*60s, *50s*/
    now.tv_sec = (4 * 86400) + (23 * 3600) + (58 * 60) + 50;
    now.tv_usec = 0;

    rc = settimeofday(&now, NULL);
    if (rc == 0) {
        printf("Heure initialisee avec succes !\n");
    } else {
        printf("Erreur : settimeofday a echoue (verifiez votre BSP).\n");
        return -1;
    }
    /* On affiche l'heure qui s'écoule */
    while(1) {
        // Récupération de l'heure actuelle mise à jour par l'Interval Timer
        gettimeofday(&tv_verif, NULL);
        printf("Temps actuel : %ld secondes (%ld jours passes)\n",
               tv_verif.tv_sec, tv_verif.tv_sec / 86400);
        sleep(1); // Pause de 1 seconde (utilise les ticks de l'Interval Timer)
    }
    return 0;
}
```

1

**gettimeofday()
settimeofday()**

Fonctions UNIX

2

**struct timeval {
 time_t tv_sec;
 suseconds_t tv_usec;
}**
Structure de données

Récapitulatif — Toutes les fonctions HAL TIMER

Fonction	Signature	Rôle	Famille
alt_alarm_start()	(alarm*, nticks, cb, ctx) → int	Lance une callback périodique	Alarme
alt_alarm_stop()	(alarm*) → void	Arrête la callback	Alarme
alt_ticks_per_second()	(void) → alt_u32	Ticks par seconde du timer	Alarme
alt_nticks()	(void) → alt_u32	Ticks écoulés depuis reset	Alarme
alt_timestamp_start()	(void) → int	Démarre le compteur continu	Timestamp
alt_timestamp()	(void) → alt_u32	Valeur courante (ticks)	Timestamp
alt_timestamp_freq()	(void) → alt_u32	Fréquence compteur (Hz)	Timestamp
gettimeofday()	(timeval*, tz*) → int	Temps absolu UNIX (sec+µs)	UNIX / HAL

Configuration BSP — system.h

```
#define ALT_SYS_CLK          INTERVAL_TIMER  /* none = désactivé */  
#define ALT_TIMESTAMP_CLK   INTERVAL_TIMER  /* none = désactivé */  
#define ALT_MAX_FD          32              /* compteur 32 bits */
```

ALT_SYS_CLK

- ▶ Pointe vers le timer pour les alarmes
- ▶ Active `alt_alarm_*` et `alt_ticks()`
- ▶ Requis pour `gettimeofday()` et `times()`
- ▶ Configuré via BSP : `hal.sys_clk_timer`

ALT_TIMESTAMP_CLK

- ▶ Pointe vers le timer dédié aux mesures
- ▶ Active les fonctions `alt_timestamp_*`
- ▶ Mettre 'none' si non utilisé
- ▶ Configuré via BSP : `hal.timestamp_timer`